

ICS Lab 1: Data Lab 实验报告

姓名：周裕博 学号：25803050041

课程：计算机系统基础 日期：2026 年 10 月

一、实验目的

本实验在严格的编码限制下，用位运算和 IEEE 754 位级操作实现整数与单精度浮点数相关的函数，目的是：

- 理解二进制补码、掩码、移位、溢出和舍入等底层机制；
- 学会用有限的运算符（! ~ & ^ | + << >>）表达逻辑，用位运算替代分支；
- 掌握单精度浮点数的符号位、阶码、尾数拆解与 round-to-nearest-even 舍入。

二、实验环境与规则

- 环境: Ubuntu Linux (x86-64), gcc -m32 -fwrapv -O -Wall, make all 生成 btest / ishow / fshow;
- 整数题只允许常量 0-0xFF、局部变量、一元 ! ~、二元 & ^ | + << >>, 禁止分支、比较、类型转换、宏和其他数据类型; P2 进一步只允许 ~ &;
- 浮点题允许 int/unsigned、任意整数常量、控制流与比较, 但禁止浮点类型、浮点运算与类型转换;
- 每题有最大操作数限制, 由 check_ops.py (内部调用 dlc) 检查, btest 验证结果。

三、整数题思路 (P1-P14)

P1 signMask. 直接 $1 \ll 31$ 得到最高位掩码 $0x80000000$, 仅用 1 个运算符。

P2 bitXor. 只用 ~ 与 &。先由德摩根律构造或: $x | y = \sim(\sim x \& \sim y)$, 再用 $x \wedge y = \sim(x \& y) \& \sim(x | y)$, 即 $\sim(x \& y) \& \sim(\sim x \& \sim y)$ 。

P3 negativePart. $x \gg 31$ 是“全 0 / 全 1”的符号掩码。负数时与 $-x = (\sim x) + 1$ 相与, 非负时掩码为 0, 结果为 0。

P4 copyByteWithin. 用 $0xFF \ll (dst \ll 3)$ 定位目标字节并取反清除, 再把源字节 $(x \gg (src \ll 3)) \& 0xFF$ 左移到目标位置后按位或。dst << 3 即 $dst * 8$ 。

P5 logicalShift. 算术右移会补符号位, 因此需要掩码 $msk = (1 \ll (32 - n)) - 1$ 保留低 $32 - n$ 位。n = 0 时令 $n = 32$, 此时掩码为 0, 并用 $x \ll 0$ 补回, 避免非法的“移位 32”。

P6 swapNibblePairs. 用掩码 $0xF0F0F0F0$ 分别取每字节的低、高半字节, 低半字节左移 4 位与高半字节右移 4 位合并。右移虽是算术右移, 但 & msk 已滤掉符号位。

P7 secondLowestZeroBit. $(\sim x) \& (x + 1)$ 能取出最低的 0 位: $x + 1$ 把尾部连续的 1 清零并把最低 0 置 1, $\sim x$ 恰好选出该位。先取出最低 0 位并把它置 1 ($x |= lowbit$), 再对新的 x 求一次即得次低 0 位; 全 1 时自然得到 0。

P8 oddParity. 先用 $x \wedge y = \sim(x \& y) \& (x \mid y)$ 把高 16 位、高 8 位异或折叠到低位，再用 $x + (x \gg 1) + \dots + (x \gg 7)$ 错位累加各 bit 的贡献，取最低位并结合符号位得到 1 的个数奇偶，最后取反以满足“偶数个 1 返回 1”。

P9 rotateRightBits. 先 $n \&= 0x1F$ 使位数按 32 取模。右移 n 位后用掩码保留低 $32 - n$ 位，再左移 $32 - n$ 位把低 n 位送到高位，两者按位或； $n = 0$ 同样特殊处理。

P10 roundEvenPow2. 取商的最低位 $(x \gg n) \& 1$ 。计算 $x + 2^{(n-1)}$ ：商为奇 ($\text{bit} = 1$) 时加满半个单位，商为偶时少加 1；恰好半途时前者进位到偶倍、后者退位到偶倍，实现 round-half-even，最后用掩码清除低 n 位。

P11 midpointTowardFirst. 为避免 $x + y$ 溢出，先算 $\text{avg} = (x \gg 1) + (y \gg 1)$ ，余数和为 $(x \& 1) + (y \& 1)$ 。用符号位判断大小关系 $\text{gr} = [x \geq y]$ ；当精确中点为半整数时由 gr 决定进位方向，使结果偏向 x 。返回 $\text{avg} + (((x \& 1) + (y \& 1) + \text{gr}) \gg 1)$ 。

P12 isBetweenEitherOrder. 实现一个跨符号安全的“ $\geq$ ”比较器：异号时正者较大，同号时差值不会溢出，可直接取差的符号位。据此算出 $a \leq x$ 、 $x \leq b$ 、 $b \leq x$ 、 $x \leq a$ 四个布尔值，返回 $(a \leq x \&\& x \leq b) \mid\mid (b \leq x \&\& x \leq a)$ 。

P13 mul5Sat. 取绝对值 $|x| = (x \wedge \text{msk}) - \text{msk}$ （代码用符号掩码选择 x 或 $(\sim x) + 1$ ），与阈值 $0x1999999A = \lfloor \text{INT_MAX} / 5 \rfloor + 1$ 比较：小于阈值时 $x * 5 = x + (x \ll 2)$ ，否则按符号饱和到 $\text{INT_MAX} / \text{INT_MIN}$ 。

P14 classifyAdd3. 把每个数拆成高 24 位 ($\gg 8$) 与低 8 位 ($\& 0xFF$)。低 8 位之和产生进位 $\text{carry} = (1x + 1y + 1z) \gg 8$ ，于是 $\text{res} = (x \gg 8) + (y \gg 8) + (z \gg 8) + \text{carry}$ 正好是精确和除以 256 的向下取整。将 $\text{res} \gg 23$ 后：结果为 1 表示正溢出，0 或 -1 表示未溢出， ≤ -2 表示负溢出；再用 $\text{res} + 1$ 、 $!\text{res}$ 区分出 -1、0 两种情况，输出 $1 / 0 / -1$ 。

四、浮点题思路 (P15–P17)

浮点题统一把 uf 拆成符号 S 、阶码 E 、尾数 M ($S = uf \gg 31$, $E = (uf \gg 23) \& 0xFF$, $M = uf \& 0x7FFFFFFF$)，再分情况处理。

P15 floatScaleThreeHalves. 目标是 $f \times 3/2$ ，采用 round-half-even。

- $E = 0$ (零或非规格化)：值为 $M \cdot 2^{-149}$ ，乘 $3/2$ 得 $3M/2$ 。对 M 做 round-half-even 得到 num 后直接返回 $\text{num} \mid (S \ll 31)$ ， num 本身即完成了非规格化到规格化的编码。
- $E = 0xFF$ ：NaN 或无穷，原样返回。
- 规格化：令 $\text{num} = M \mid 0x800000$ ，计算 $\text{num} + (\text{num} \gg 1)$ ，若 num 为奇（半途）且结果最低位为奇则加 1，实现 round-half-even；若尾数溢出到第 25 位，则阶码加 1、尾数右移一位并再次舍入，阶码达到 $0xFF$ 时返回无穷。

P16 floatRoundEven. 把浮点值舍入到最近的整数（半途取偶），NaN/无穷原样返回。

- $E = 0$ 或 $E \leq 125$ ：值必小于 0.5，舍入为 0，但保留符号位返回 $S \ll 31$ ；
- $E \geq 150$ ：值已是整数 ($\geq 2^{23}$)，原样返回；

- 其余 ($126 \leq E \leq 149$) 令 $\text{bit} = 23 - (E - 127)$ 为小数位数, 把有效位 $\text{num} = M \mid 0x800000$ 的低 bit 位抽出、按 2^{24} 对齐得到 decimal , 与半值 $0x800000$ 比较并按 round-half-even 决定舍入方向, 最后用“消去隐含前导 1”的写法重建指数与尾数。

P17 float_i2f. 取 $\text{absx} = |x|$ 与符号。找到最高位 i , num 保存去掉最高位后的低位部分: 若 $i > 23$, 右移 $i - 23$ 位为尾数, 并保留被移出的 remainder , 与半程比较 (相等时看尾数最低位) 做 round-half-even; 若尾数进位到 2^{23} , 则 i 加一并调整尾数。若 $i \leq 23$ 则精确左移 $23 - i$ 位。返回 $\text{sign} \mid ((i + 127) << 23) \mid \text{num}$ 。

五、位统计题思路 (P18–P19)

P18 bitCount. 采用 SWAR 并行统计: 依次用 $0x55555555$ 、 $0x33333333$ 、 $0x0F0F0F0F$ 、 $0x00FF00FF$ 、 $0x0000FFFF$ 作掩码, 把相邻 1、2、4、8、16 位的计数两两相加 $((\text{num} \& \text{msk}) + ((\text{num} >> k) \& \text{msk}))$, 最终得到 32 位中 1 的个数。所有大掩码均由 $0x55$ 等小常量逐级拼接而成。

P19 bitReverse. 分治法交换位块: 依次交换高/低 16 位、相邻 8 位、4 位、2 位、1 位。掩码序列由上一掩码异或左移生成: $0x0000FFFF \rightarrow 0x00FF00FF \rightarrow 0x0F0F0F0F \rightarrow 0x33333333 \rightarrow 0x55555555$, 每步用 $((x \& \text{msk}) << k) \mid ((x >> k) \& \text{msk})$ 完成交换。

六、测试结果

`make clean` && `make all` 构建成功; `./check_ops.py bits.c` 输出 “All 19 functions passed operator checks.”; `./btest` 全部函数 Errors 为 0, 总分为 110/110。各题操作数与分值如下表。

编号	函数	分值	操作数	上限
P1	signMask	1	1	2
P2	bitXor	2	7	8
P3	negativePart	3	4	6
P4	copyByteWithin	4	10	12
P5	logicalShift	4	9	20
P6	swapNibblePairs	4	11	18
P7	secondLowestZeroBit	4	7	8
P8	oddParity	5	30	56
P9	rotateRightBits	5	9	16
P10	roundEvenPow2	5	10	24
P11	midpointTowardFirst	5	26	32
P12	isBetweenEitherOrder	7	38	48
P13	mul5Sat	7	30	30
P14	classifyAdd3	7	17	52
P15	floatScaleThreeHalves	7	31	60
P16	floatRoundEven	10	45	65
P17	float_i2f	10	39	40
P18	bitCount	10	36	40
P19	bitReverse	10	34	34

表 1 各题分值、实际操作数与上限（来自 check_ops.py）

七、总结

- 整数题的核心是“用掩码和移位代替判断”：符号掩码 $x \gg 31$ 统一处理正负，掩码配合 $\sim$ 实现清零与置位，避免了一切控制流与比较运算符。
- 在受限运算符下，德摩根律是构造 $\wedge$ 、 $\vee$ 的关键；SWAR 与分治思想则把逐位操作变成并行的位级加法和交换，兼顾正确性与操作数上限。
- 溢出判断不能依赖结果本身，应先降精度（如拆成高低位、取绝对值）再比较，例如 mul5Sat、classifyAdd3、midpointTowardFirst。
- 浮点题需要完整处理符号、阶码、尾数以及零、非规格化数、无穷、NaN 等特殊值，舍入统一采用 round-to-nearest-even，并注意半途判定与尾数进位引起的阶码变化。

本次实验全部 19 题通过规则检查与正确性测试，加深了对计算机中整数与浮点数位级表示的理解。